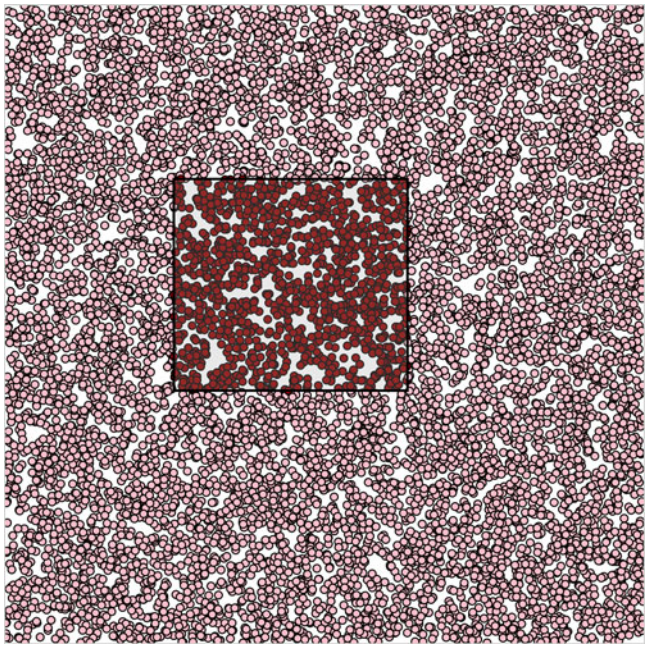


Figure 11.14 Quadtree-optimized selection used with a dataset of 10,000 points

Listing 11.15 Creating a quadtree from xy data

```
var quadtree = d3.geom.quadtree()
    .extent([[0,0], [500,500]])
    .x(function(d) {return d.x})
    .y(function(d) {return d.y});

var quadIndex = quadtree(sampleData);
```

Accessors pointed at our data's xy format

We need to define the bounding box of a quadtree as an array of upper-left and lower-right points.

After creating a quadtree, we create the index by passing our dataset to it.

After you create a quadtree and use it to create a quadtree index dataset like we did with `quadIndex`, you can use that dataset's `.visit()` function for quadtree-optimized searching. The `.visit()` functionality replaces your test in a new brush function, as shown in listing 11.16. First, I'll show you how to make it work in listing 11.16. Then, I'll show you that it *does* work in figure 11.14, and I'll explain *how* it works in detail. This isn't the usual order of things, I realize, but with a quadtree, it makes more sense if you see the code before analyzing its exact functionality.

Listing 11.16 Quadtree-optimized xy brush selection

```
function brushed() {
  var e = brush.extent();

  d3.selectAll("circle")
     .style("fill", "pink")
     .each(function(d) {d.selected = false})
```

Sets all circles to pink, and gives each a selected attribute to designate which are in our selection